

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

C02: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour | Total Marks: 50

Annexure A — Scenario Based Assignment

Code of Conduct:

I **PERAM VIVEKANANDA REDDY(192372330)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

vivek

Question 1 — Drone-Based Emergency Delivery in a Flood-Affected Region

Scenario recap: An AI-controlled drone must deliver emergency medicines through a flood-hit, dynamic environment. It has a heuristic (straight-line distance), partial real-time information about blocked routes, and variable terrain/weather costs. Speed and safety both matter.

i. Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal according to the heuristic $h(n)$ alone — it ignores the cost already incurred, $g(n)$. In this scenario, at every junction the drone would simply fly toward whichever next waypoint has the smallest straight-line distance to the delivery point.

Strengths: It is fast and memory-light, which suits the urgency of the mission — it produces a route almost immediately because it does very little bookkeeping of path cost.

Weaknesses under uncertainty: Because it never reconsiders accumulated cost, it can be lured toward a route that looks close on a straight line but is actually blocked by floodwater or has a high weather penalty. It is neither complete nor optimal in general graphs, so under partial and changing information it can commit to a route that turns out to be a dead end, forcing costly backtracking exactly when time is critical.

ii. Behaviour of A* Search

A* evaluates nodes using $f(n) = g(n) + h(n)$, combining the actual cost so far with the heuristic estimate of the remaining distance. For the drone, $g(n)$ would capture the real terrain/weather cost accumulated along the path, while $h(n)$ remains the straight-line estimate to the destination.

This balance means A* will not blindly chase the node that merely looks close; it also weighs how expensive/risky the path taken to reach that node has been. As long as the heuristic is admissible (never overestimates true remaining cost) and consistent, A* guarantees the least-cost path is found, and it still focuses its search efficiently rather than exploring uniformly in every direction like uninformed search.

iii. Impact of Heuristic Accuracy on Both Algorithms

- If the heuristic is highly accurate (close to true remaining cost), GBFS tends to behave well and its greedy choices coincide with good routes; A* also becomes very efficient because $f(n)$ closely tracks reality, pruning irrelevant branches early.
- If the heuristic is inaccurate or misleading (e.g., ignores flooded zones because it is purely geometric), GBFS can be seriously misled, since heuristic error is its only guide — it has no cost memory to compensate.
- A* is more robust to heuristic inaccuracy because $g(n)$ continues to reflect the real cost paid; a poor heuristic mainly costs A* extra computation (more nodes explored), not correctness — provided the heuristic still stays admissible. If the heuristic overestimates cost, however, A* can lose its optimality guarantee.

iv. Effect of Dynamic Changes (Blocked Paths) on Search Performance

Both algorithms assume a largely static search space unless explicitly adapted. When a road/route suddenly becomes inaccessible:

- GBFS may already have committed to that direction (since it does not track cost) and can waste significant time before realizing the route is a dead end.
- A* is somewhat better positioned because its priority queue keeps alternative, partially-explored paths ranked by real $f(n)$ values, so it can more gracefully switch to another frontier node — but a plain A* still needs to re-plan (e.g., via Repeated A*, D*, or D* Lite variants) to properly incorporate new blockage information without restarting from scratch.
- In general, sudden changes increase node re-expansion, add latency, and can force either algorithm to discard partially built plans — this is the core challenge of searching in dynamic, partially observable environments.

v. Recommendation and Justification

A* is the more suitable algorithm for this scenario (ideally an incremental/real-time variant such as D* Lite for the dynamic re-planning aspect). Justification:

- Optimality and safety: A* guarantees the least-cost (and by extension safest/least risky, if risk is encoded in edge cost) path given an admissible heuristic, whereas GBFS offers no such guarantee — an important consideration when delivering emergency medicine.
- Real-time constraints: although A* is more computationally demanding than GBFS, it remains far more efficient than uninformed search, and with a well-tuned heuristic (e.g., incorporating known flood-risk zones) it can meet real-time deadlines while still respecting real path costs.
- Robustness: A*'s use of $g(n)$ makes it more resilient to heuristic imperfection and better able to recover from blocked-path updates than GBFS, which has no mechanism to correct a bad early decision.

Overall, the marginal extra computation of A* is a reasonable trade-off for the large gain in reliability and safety that a purely greedy approach cannot offer in a life-critical, uncertain environment.

Question 2 — Smart City Traffic Signal Optimization

Scenario recap: An AI system must set traffic-signal timings across many intersections to minimise waiting time, fuel consumption, and congestion, in a search space that is huge, dynamic, and prone to local optima.

Formulation as an Optimization Problem

This is best framed as a continuous/combinatorial local-search (optimization) problem rather than a path-finding problem:

- State: a complete assignment of green/red signal durations and phase offsets to every intersection at a given time.
- Objective function: a weighted combination of total waiting time, fuel consumption, and congestion level (to be minimised) — essentially an energy/cost function over the space of all possible timing configurations.
- Neighborhood/move: small adjustments to one or more signals' timings (e.g., +/- a few seconds of green time at one junction).

Traditional systematic search (BFS/DFS/A* over discrete states) is inefficient here because the state space is astronomically large (every intersection has a continuous timing parameter, and there are hundreds of intersections), and there is no natural notion of a single 'goal state' to search toward — this is a case of finding the best configuration among many, not reaching one known target, so local-search / optimization methods that iteratively improve a candidate solution are far more suitable than tree/graph search.

i. Hill Climbing in this Scenario

Hill Climbing would start from some initial timing configuration and repeatedly move to a neighbouring configuration only if it improves the objective (e.g., reduces average waiting time), stopping when no neighbour improves further.

Limitations: it only looks at immediate neighbours and always moves 'uphill' (or downhill, for minimisation), so it can easily get trapped in a solution that is only locally good, it has no mechanism to escape once no adjacent move helps, and its final result depends heavily on the starting configuration.

ii. Local Maxima, Plateaus, and Ridges (Real-World Interpretation)

- Local maxima: a signal-timing pattern where every intersection's timing seems balanced for its own approach, yet city-wide congestion is still far from optimal — Hill Climbing stops here even though better overall configurations exist elsewhere.

- Plateaus: many neighbouring timing tweaks produce almost identical waiting-time scores (e.g., shifting green time by a second or two makes no measurable difference), so the search has no clear direction to move and can wander or stall.
- Ridges: improvement is possible, but only by changing several intersections' timings together in a coordinated way (e.g., a 'green wave' along a corridor); changing one signal at a time (Hill Climbing's usual move) may look like it worsens things even though a joint change would help, so the search misses the improving direction.

iii. Simulated Annealing and Genetic Algorithms as Solutions

Simulated Annealing: occasionally accepts a worse configuration with a probability that decreases over time (as the 'temperature' cools). This lets it escape local maxima and cross plateaus early in the search, while converging toward a strong solution as it cools — well suited to gradually refining signal timings while avoiding premature convergence.

Genetic Algorithms: maintain a population of candidate timing plans, combine (crossover) good ones, and randomly mutate others, evolving city-wide traffic plans over generations. This population-based search naturally explores several regions of the vast search space in parallel, making it more likely to find coordinated (ridge-type) improvements than single-point methods, and is well suited to distributed, scalable optimisation across many intersections.

iv. Recommended Approach and Justification (Scalability and Adaptability)

A Genetic Algorithm (optionally hybridised with Simulated Annealing for local refinement) is recommended:

- Scalability: GAs naturally parallelise across intersections/sub-regions and populations, and can be distributed across computing nodes to handle hundreds of intersections.
- Adaptability: because traffic patterns shift continuously (rush hour, incidents, weather), a population of candidate solutions can be continuously re-evaluated and evolved online rather than restarting a single-point search from scratch, and Simulated Annealing's gradual cooling can be re-triggered for rapid local re-tuning when conditions change.
- Avoiding entrapment: both methods are explicitly designed to escape the local maxima, plateaus, and ridges that would trap simple Hill Climbing, which is essential given the objective landscape here is known to be highly non-convex.

Question 3 — Autonomous Mars Rover Exploration

Scenario recap: A rover must navigate unknown Martian terrain, avoid hazards, work with limited sensing, and make decisions without a complete map, updating its knowledge as it explores.

i. Why an Online Search Agent, Not Offline Search

Offline search algorithms compute a complete solution/path before execution begins, assuming the whole environment (map) is known in advance. The rover does not have this luxury — the terrain is unknown until sensed, so any pre-computed 'complete' plan could be invalidated the moment a hidden crater or rock is discovered. An online search agent instead interleaves computation with action: it takes an action, observes the real outcome, updates its knowledge of the environment, and re-plans — which is the only viable approach when the map is not available up front.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover only perceives its immediate sensor range, so it cannot know in advance whether the path ahead is safe; it must make locally-informed decisions that could later prove sub-optimal once more of the terrain is revealed.
- Risk of irreversible actions: unlike a simulated search where you can 'backtrack' freely, physically returning from a hazardous position (e.g., near a crater edge) may be costly, slow, or impossible, so exploration must be conservative.
- Dynamic conditions: dust storms, shifting sand, or lighting changes can alter what is safe or even alter the terrain itself, meaning previously gathered knowledge can become outdated and must be continuously revalidated.
- Computation under resource constraints: the rover has limited onboard compute/power and communication delay with Earth (light-time lag), so it cannot rely on a distant control centre for every decision and must reason locally and efficiently.

iii. How the Rover Builds and Updates its Internal Model

As the rover moves, its sensors (cameras, LIDAR, etc.) capture local terrain data, which is fused into an incrementally built internal map (e.g., an occupancy grid or graph of visited/adjacent cells marked safe, hazardous, or unknown). Each new observation updates the cost/heuristic estimates and hazard labels for nearby regions. This internal model is used to choose the next action online (e.g., using an online variant of A* or LRTA* — Learning Real-Time A*), and is continuously refined so that later decisions benefit from everything learned during earlier exploration, gradually converging as more of the terrain becomes known.

iv. Online Search vs Uninformed and Informed (Offline) Strategies

- Uninformed offline search (BFS/DFS): requires a fully known map ahead of time and explores blindly without any goal-direction; entirely unsuitable here since no such complete map exists and blind exploration would be dangerously inefficient over hazardous terrain.
- Informed offline search (A*): would be excellent if a complete, accurate map and heuristic were available in advance, guaranteeing optimal paths — but it cannot execute plans through terrain that isn't actually known before the journey starts.
- Online search (e.g., LRTA*): combines the goal-directed efficiency of heuristic search with the ability to act, observe, and re-plan as new information arrives, making it the only category of the three that fits a genuinely unknown, partially observable environment; the trade-off is that its resulting path is generally not guaranteed to be globally optimal, since decisions are made with incomplete knowledge.

v. Justification of the Most Suitable Approach

An online, informed search strategy (an LRTA*-style approach, using a heuristic based on estimated distance to the sample-collection goal, updated as terrain is sensed) is the most suitable choice. It directly addresses uncertainty (decisions are revisited as new sensor data arrives), adaptability (the internal model and future action choices update continuously rather than assuming a static world), and safety (hazards are avoided as soon as they are perceived rather than only being handled by a plan made in ignorance of their existence) — properties that neither pure offline uninformed nor offline informed search can offer in this setting.

Question 4 — University Exam Timetable as a CSP

Scenario recap: An AI system must generate exam timetables subject to no overlapping exams for a student, limited exam halls, precedence between certain subjects, faculty availability, and special accommodations, with complexity growing with courses/students.

Formulation as a Constraint Satisfaction Problem (CSP)

i. Variables, Domains, and Constraints

Variables: one variable per course/exam to be scheduled, e.g., $X_1, X_2, \dots, X_n$, each representing 'the (time slot, hall) assigned to that course's exam.'

Domains: for each variable X_i , the domain is the set of all valid (time slot, hall) pairs available during the exam period — e.g., {Day1-9AM-HallA, Day1-9AM-HallB, Day1-2PM-HallA, ...}. Domains may be pruned in advance for courses that need special halls (e.g., labs) or accommodations (e.g., a quiet/extra-time room).

Constraints: these fall into a few natural categories —

- Unary constraints: a course requiring a special accommodation room restricts its own domain to only slots in that room type.
- Binary 'no overlap' constraints: for any two courses sharing at least one common student, their assigned time slots must differ ($X_i.time \neq X_j.time$).
- Capacity constraints: for any given (day, time) slot, the total students examined across all courses scheduled then must not exceed total hall capacity for that slot; equivalently, no two courses assigned the same hall at the same time ($X_i.(time, hall) \neq X_j.(time, hall)$ if halls collide).
- Precedence constraints: if subject A must be examined before subject B, then $X_a.time < X_b.time$.
- Faculty availability constraints: a course's exam slot must fall within times its invigilating faculty is available.

ii. Backtracking Search in this Scenario

Backtracking search assigns exams to slots one variable (course) at a time; after each assignment it checks whether all constraints involving already-assigned variables are still satisfied. If a partial assignment violates a constraint (e.g., two courses with common students end up in the same slot), the algorithm backtracks — undoing the most recent assignment(s) and trying a different value from the domain — continuing this systematic trial-and-error until either a complete valid timetable is found or the search proves no solution exists with the current setup. Variable/value ordering heuristics (e.g., choosing the most constrained course first, the 'minimum remaining values' heuristic) are typically used to make this efficient in practice.

iii. Constraint Propagation (e.g., Forward Checking)

Constraint propagation reduces wasted search effort by removing values from the domains of not-yet-assigned variables as soon as they become inconsistent with current assignments, rather than discovering the conflict only after a full assignment attempt. In forward checking specifically, whenever a course is scheduled, the algorithm immediately removes that (time, hall) combination — and any combination that would violate a no-overlap/precedence/capacity constraint with it — from the domains of all courses that share a constraint with it. This lets the algorithm detect a dead end far earlier (a variable left with an empty domain signals immediate failure), substantially reducing the number of backtracks needed and making the search scale far better as the number of courses/students grows.

iv. Real-World Challenges and Justification of the Best Strategy for Scalability

- Challenges: the constraint graph becomes extremely dense as course and student numbers grow (many courses share students), special accommodations and faculty constraints add extra layers of unary/binary restrictions, and hard capacity limits on halls can make some naïve orderings fail late in the search, wasting computation.
- Best strategy: backtracking search combined with forward checking (or the stronger arc-consistency technique, AC-3) plus good heuristics — Minimum Remaining Values (choose the most constrained course next) and Least Constraining Value (try the slot/hall that rules out the fewest options for others) — offers the best balance of correctness and scalability. This combination catches conflicts early, focuses effort on the hardest-to-place courses first, and scales far better to large universities than plain backtracking or brute-force enumeration, while still guaranteeing a valid timetable is found whenever one exists.

Question 5 — Real-Time Strategy Game AI (Minimax & Alpha-Beta Pruning)

Scenario recap: An AI opponent must compete with human players in a real-time strategy game, deciding within strict time limits over an extremely large game tree of possible moves.

i. How Minimax Models this Problem

Minimax treats the game as a two-player, zero-sum contest represented by a game tree: each node is a game state, and edges are possible moves. The AI (maximizing player) tries to choose moves that maximize its evaluated outcome, while assuming the opponent (minimizing player) will always choose moves that minimize the AI's outcome (i.e., play optimally against it). The algorithm recursively evaluates the tree from the leaves (terminal or depth-limited states, scored by an evaluation function) upward, backing up the minimum at opponent-move levels and the maximum at AI-move levels, so the root move chosen is the one that is best for the AI under worst-case (optimal) opposition.

ii. Computational Challenges of Large Game Trees

- Exponential growth: the number of nodes grows roughly as b^d , where b is the branching factor (number of legal moves at each state) and d is the search depth — in a real-time strategy game with many units and actions, b can be very large, making full-depth search computationally infeasible.
- Strict time limits: a real-time game requires a decision within a small time budget, so exhaustively evaluating the full tree is impossible; the algorithm must be depth-limited and use an evaluation function at cut-off nodes instead of true terminal values.
- Memory and evaluation cost: storing/evaluating large numbers of intermediate states consumes memory and CPU, and a poorly designed evaluation function can itself be slow to compute at scale.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning keeps track of two bounds while traversing the tree: alpha (the best value the maximizing player can guarantee so far) and beta (the best value the minimizing player can guarantee so far). While exploring a node's children, if a branch is found where the minimizing player could force a result worse (for the maximizer) than a value already guaranteed elsewhere (i.e., $\beta \leq \alpha$), the remaining sibling branches at that node can be safely skipped ('pruned') because a rational opponent would never let the game reach a state that helps the maximizer more than an already-available alternative.

This does not change the final decision (it still returns the same result as full Minimax) but can, in the best case (with good move ordering), reduce the effective branching factor from b to roughly $\sqrt{b}$, allowing the search to reach nearly double the depth in the same time budget — a critical gain when operating under real-time constraints on large game trees.

iv. Designing an Evaluation Function

Since full-depth search is infeasible, a depth-limited Minimax needs a heuristic evaluation function to score non-terminal states. For a real-time strategy game, reasonable factors to combine (typically as a weighted sum) include:

- Material/resource advantage: difference in unit count, unit strength, and resources (gold/ore/etc.) between the AI and the opponent.
- Territorial control: number/value of map regions, bases, or strategic points controlled.
- Positional strength: unit positioning relative to choke points, high ground, or defensive structures.
- Economic growth potential: production capacity, expansion rate, and resource-gathering efficiency.
- Threat/vulnerability assessment: exposure of key units/structures to attack, and immediate tactical threats.

These factors are chosen because they each correlate with long-term winning chances and can be computed quickly from the current game state, which is essential since the evaluation function will be called at a very large number of nodes under a tight time budget.

v. Strategy for Balancing Optimality and Real-Time Performance

- Iterative deepening: run Minimax with Alpha-Beta at increasing depth limits, always keeping the best move found so far, so the AI can be interrupted at any time (when the time budget runs out) and still return a reasonable decision.
- Good move ordering: evaluate the most promising moves first (e.g., using a quick heuristic or results from previous iterations) to maximize the amount of pruning Alpha-Beta achieves.
- Transposition tables: cache previously evaluated states to avoid recomputation when the same state is reached via different move orders, which is common in strategy games.
- Selective/adaptive depth: search more deeply along forcing or high-stakes lines and more shallowly elsewhere, rather than exploring the whole tree uniformly.

Together, these techniques let the AI trade a small amount of theoretical optimality for a large gain in real-time responsiveness, producing strong, timely decisions even though the true game tree is far too large to search exhaustively.

Summary Table

Question	Core Algorithm(s)	Key Recommendation
Q1	Greedy Best-First Search vs A*	A* (with dynamic re-planning, e.g., D* Lite) for optimality and safety
Q2	Hill Climbing vs Simulated Annealing / Genetic Algorithms	Genetic Algorithm (hybrid with Simulated Annealing) for scalability and adaptability
Q3	Offline vs Online (LRTA*) search	Online, informed search agent for uncertainty and safety
Q4	Backtracking + Constraint Propagation (Forward Checking)	Backtracking with forward checking / AC-3 and MRV/LCV heuristics
Q5	Minimax and Alpha-Beta Pruning	Alpha-Beta with iterative deepening and move ordering for real-time play